



The six-minute pitch

A fresh recording of the console. What to say, and what to have on screen while you say it.

Nine segments, **744 words**, about **5:08** of speech in a six-minute slot — roughly **52 seconds** of pause across the eight breaks. The arc has room for a worked example and for both safety behaviours, which are the things that separate this from an auto-router.

744

words to read

5.1 min

of speech at 145 wpm

52 s

of pause to spend

Before you record

- **Camera at the start and the end, screen in between.** Face for the first segment, cut to the console for the middle, back to camera for the last two sentences.
- **Record segment by segment.** Nine short takes are far easier to fix than one long one, and each can slide to where its part of the screen actually starts.
- **Console at 100% zoom, maximised, already open.** The default seed is warmed into the image so it loads in about a second, but open it before you start rolling anyway.
- **No slides.** The decision-path row on the top card is the architecture diagram, and it has the advantage of being real. Point along it rather than cutting to a drawing.
- **Click the 40% link before you start that segment,** so the re-run happens under your first sentence rather than in silence.

The running order

Starts	Segment	Words	Speech
0:00	Who, and the problem	104	43s
0:46	How it decides	83	34s
1:23	One incident, end to end	87	36s
2:02	What it did, and what that was worth	94	39s
2:45	What it refused	90	37s
3:25	When it undoes itself, and when it stops	82	34s
4:02	Where I was wrong	77	32s
4:37	One run proves nothing	59	24s
5:05	Where the model is, and close	68	28s

The first four segments are the argument; the rest is evidence for it. If you have to lose something, lose the second half of the rollback segment or the congestion sentence in 'Where I was wrong' — both are second examples of a point already made.

The script

Bold is a number — land on it, then pause. Square brackets are stage directions, not words to read.

0:00 Who, and the problem · 104 words · ~43s

ON SCREEN You on camera. Console open behind you, or cut to it at the end.

Hi, I'm Sachin. This is RazorGuard — a payment revenue recovery system that finds failures nobody gets paged for, routes around them under written bounds, and proves what that was worth.

A gateway is not one pipe. Traffic splits across acquiring banks, methods and issuing banks, so one stream is really dozens of narrow routes. And when payments fail, it is almost never everywhere at once.

Say a route carrying four percent of volume collapses from ninety-six percent success to forty. Your fleet success rate moves **two points** — inside normal daily variation. No alert fires, and the money leaves quietly until somebody notices by hand.

0:46 How it decides · 83 words · ~34s

ON SCREEN Decision record, top card. Point along the decision-path row as you name each step — it is the architecture diagram, and it is real rather than drawn.

Three gateways — alpha, beta, gamma. Gamma starts failing for UPI from one bank. RazorGuard does not watch the fleet average; it watches every route, and asks where the failures are concentrated.

[point along the row] It finds the common factor. It checks whether another gateway is genuinely healthier. It puts that through a policy check. Only then does it move a bounded amount of traffic — and then it measures the destination.

Detect, attribute, check, act, measure. And if the destination turns out worse, put the traffic back.

1:23 **One incident, end to end** · 87 words · ~36s

ON SCREEN Same card. The signal cell, then the money-at-risk cell.

This is the worst incident in the run. A gateway outage across **24 routes**. Success rate on them fell from **93.7%** to **33.4%**, caught in **one minute**, with **Rs 64.74 L** at risk.

This is not an alert saying something went wrong. It is a decision record: what it saw, which of its own bounds it checked, and what it intends to do about it.

The cells tinted blue were measured against a control arm during the run. The plain ones are policy written before it started.

2:02 **What it did, and what that was worth** · 94 words · ~39s

ON SCREEN Scroll to 'What the system did', then up to the KPI tiles.

It moved traffic off the failing routes in **twenty-two** bounded steps, always leaving a **three percent** canary so the broken route stays observable. On this incident that recovered **Rs 34.07 L** — **53%** of what it had put at risk.

Across all eighteen incidents, **Rs 1.19 cr** net of fees.

And that figure is measured, not estimated. The same demand runs twice — once with the router off, once with it on. Both arms see bit-identical traffic, so the difference is the routing. If they ever diverge, the run refuses to report a figure at all.

The incident recovers its own figure here, not the run's. Saying the run total over one card is the error most likely to be caught.

2:45 **What it refused** · 90 words · ~37s

ON SCREEN Refused. The reconciliation table at the top, then the rules.

Recovery is not always the right move, and this is the page I would want to be asked about.

93 times it declined to move money. The page reconciles that rather than asserting it: **57** named a policy rule, **32** were rollbacks, **4** were alarms a single open breaker covered.

The case I would point at is an issuer outage. If the bank itself is failing, gamma is broken but so are alpha and beta — moving traffic between them recovers nothing. RazorGuard recognises that signature and escalates instead of shuffling.

3:25 When it undoes itself, and when it stops · 82 words · ~34s

ON SCREEN Rollbacks. Then back to Refused and point at efficacy_breaker.

32 times it moved traffic, measured the destination, found it worse than baseline, and put the traffic back. A rollback is not the system failing. It is the system working.

And this rule is the one I am proudest of. The efficacy breaker measures the realised effect of the system's own recent shifts. When they stop paying, it halts them and escalates.

An automated system that moves money has to know when to act. It also has to know when to stop.

4:02 Where I was wrong · 77 words · ~32s

ON SCREEN Settings → 'Things worth trying' → **Max shift 40%**. Click it before you start talking.

Every bound is tunable and re-runs both arms, which is how I found out I was wrong twice.

The cap on how much one action may move started at forty percent, because forty felt careful. [the page returns] Recovery falls from **1.20 crore to 71 lakh**. My caution was costing more than half of it.

And a sweep over congestion found something worse: on a fleet already at capacity, rerouting *loses* money. That finding is why the efficacy breaker exists.

4:37 One run proves nothing · 59 words · ~24s

ON SCREEN The sidebar 'Validated result' block.

One run proves nothing, so — **8 seeds**. Mean recovery **Rs 1.23 cr**, ninety-five percent interval **Rs 1.18 cr to Rs 1.28 cr**. **38.2%** of the money at risk, and **0.58 points** of fleet success rate.

Zero of 8 seeds lost money.

This is a validated simulation, not a production deployment — the world is invented and the measurement is not.

5:05 Where the model is, and close · 68 words · ~28s

ON SCREEN How it was measured, bottom row. Cut back to camera for the last two sentences.

Detection, attribution, policy and routing are deterministic and auditable. A model writes the operator note, investigates a past decision and advises on escalations — and can change none of them. An LLM that can move money is a liability.

Detect the hidden failure. Check your own bounds. Move traffic. Measure it. Undo it when you were wrong, and stop when the whole strategy stops paying.

That's RazorGuard. Thank you.

Three things not to say

- **Do not put the run's total on one incident.** The twenty-two steps on that card recovered **Rs 34.07 L**. The **Rs 1.19 cr** is all eighteen incidents together. Saying the second over the first inflates one card by more than three times, and the page underneath contradicts you.
- **Do not say every refusal carries a rule.** Of the 93, **57** named one; the rest are rollbacks and alarms an open breaker covered. The page reconciles all three, so the narration has to as well.
- **Do not call it deployed.** It is a validated simulation. The world is invented and the measurement is not — that is a stronger claim than a vague production one, and it survives being questioned.

Every number you say, and where it comes from

Interpolated when this document is built, from `bench/results/*.json` and from the same control-plane run the console renders.

Said aloud	Value	Source
Mean recovery, 8 seeds	Rs 1.23 cr	validate.py
95% interval	Rs 1.18 cr to Rs 1.28 cr	the same
Share of money at risk	38.2%	the same
Success-rate gain	+0.58pp	the same
Seeds that lost money	0 of 8	the same
This run, net of fees	Rs 1.19 cr	experiment.py
Worst incident, before	93.7%	console run
Worst incident, during	33.4%	the same
Worst incident, at risk	Rs 64.74 L	the same
Worst incident, recovered	Rs 34.07 L	the same
Worst incident, capture	53%	the same
Refusals, and how they split	93 = 57 + 32 + 4	the same